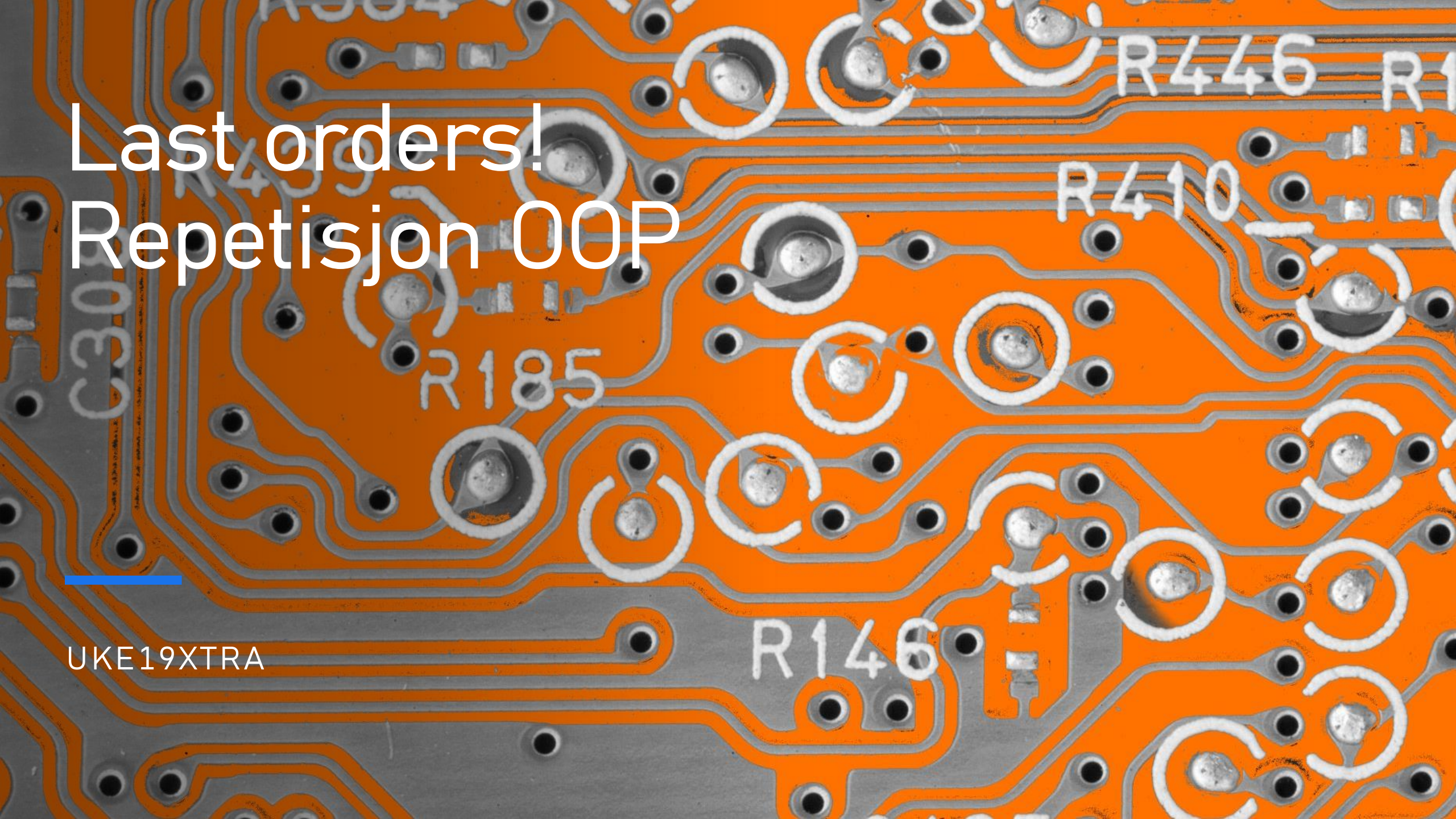




Last orders!
Repetisjon 00P

UKE19XTRA



Repetisjon

- Arv, interface , abstrakte klasser
- Unntakshåndtering
- JDBC
- Eksamensforberedelse

Arv vs Interface

Difference Between Class and Interface

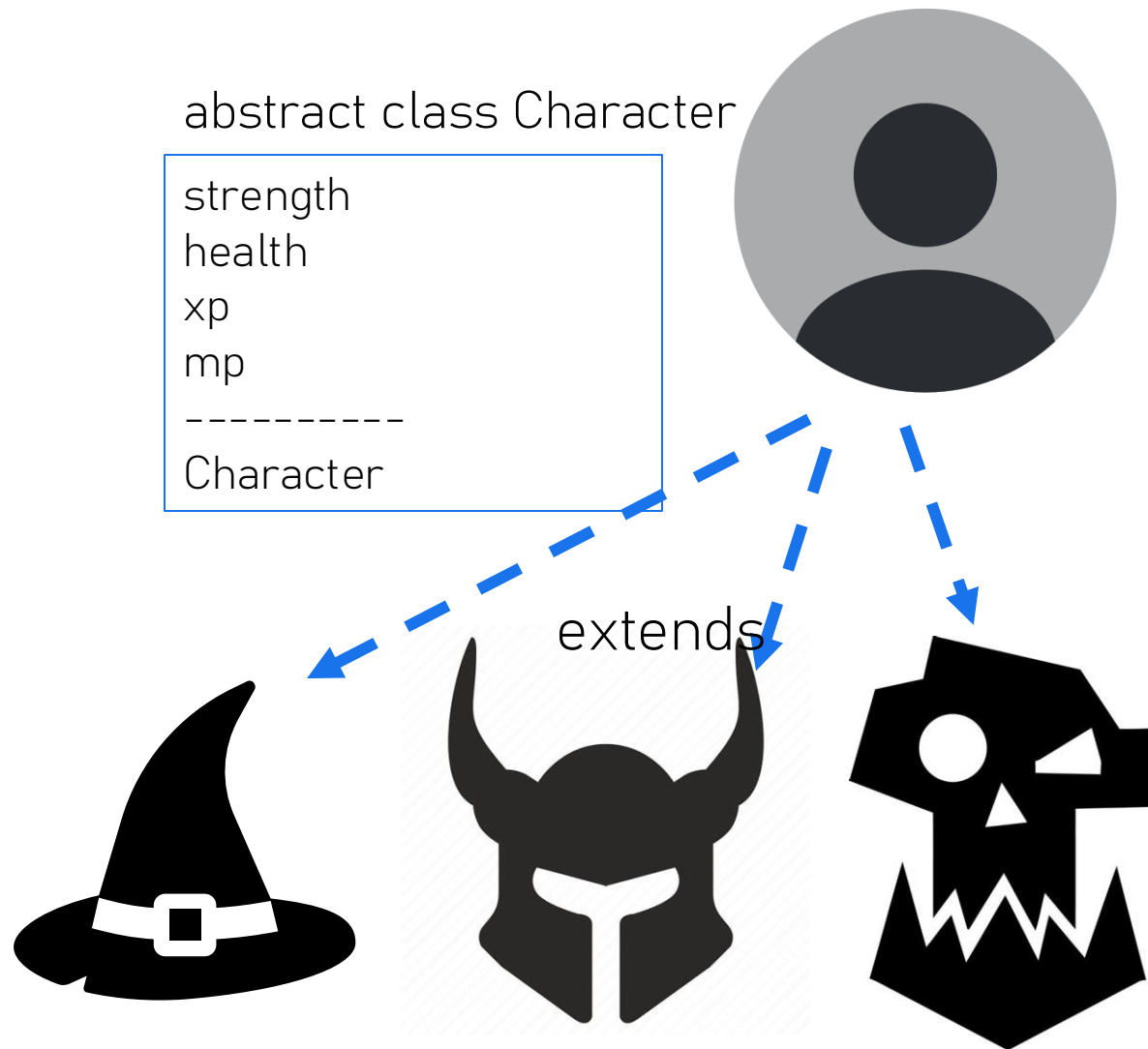
Although Class and Interface seem the same there are certain differences between Classes and Interface. The major differences between a class and an interface are mentioned below:

Features	Class	Interface
Instantiation	In class, you can create an object.	In an interface, you can't create an object
Variables	Class can have instance variables	Variables are public static final (constants only).
Methods	Class can have concrete methods	In an interface, methods are abstract by default
Inheritance	It supports single inheritance	Supports multiple inheritance
Constructors	Can have constructors.	No constructors allowed.
Access Modifiers	Supports private, protected, public, default.	In an interface, all members are public by default

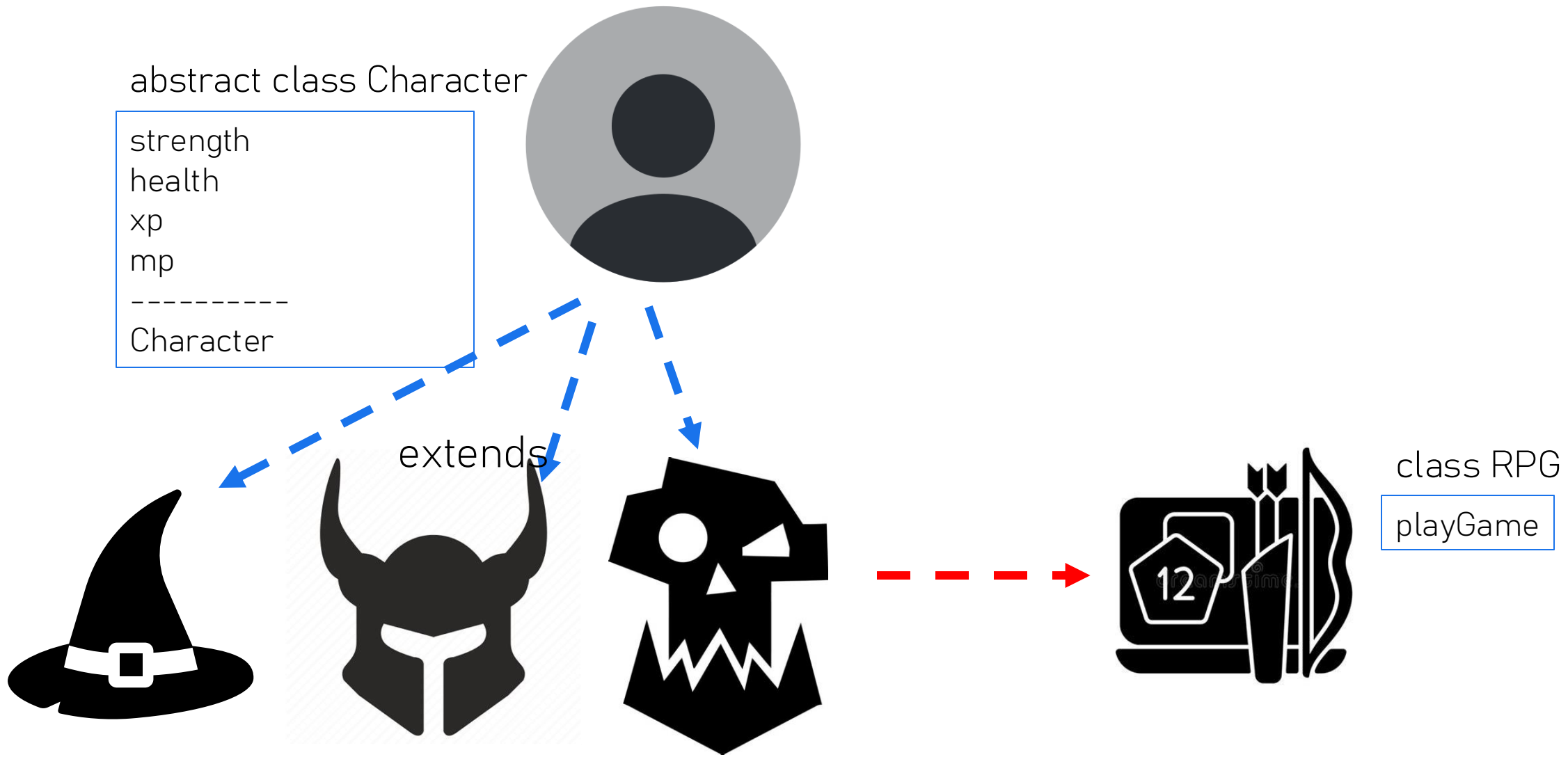
Arv vs Interface

- I arv “utvides” (extends) **en** klasse til underklasser.
- Interface “implementeres” (implements) til andre klasser.
- “Abstrakte” klasser kan ha konstruktører og metoder og instansvariabler som brukes i underklassene.
- Interfaces har ingen konstruktører. Den har “abstrakte” metoder som **må** implementeres av klasser som bruker interface.
- Hverken “abstrakte” klasser eller interfaces kan instansiere objekter.

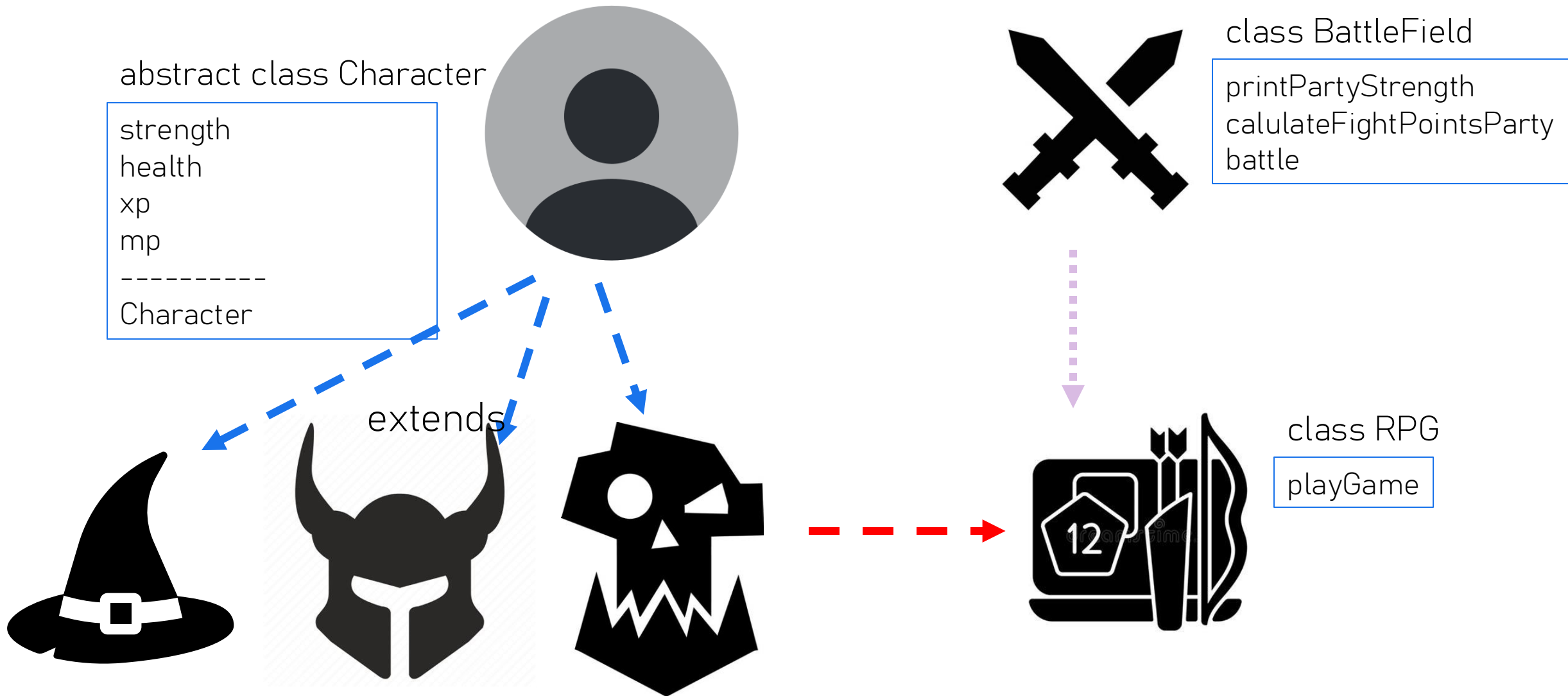
Arv vs Interface



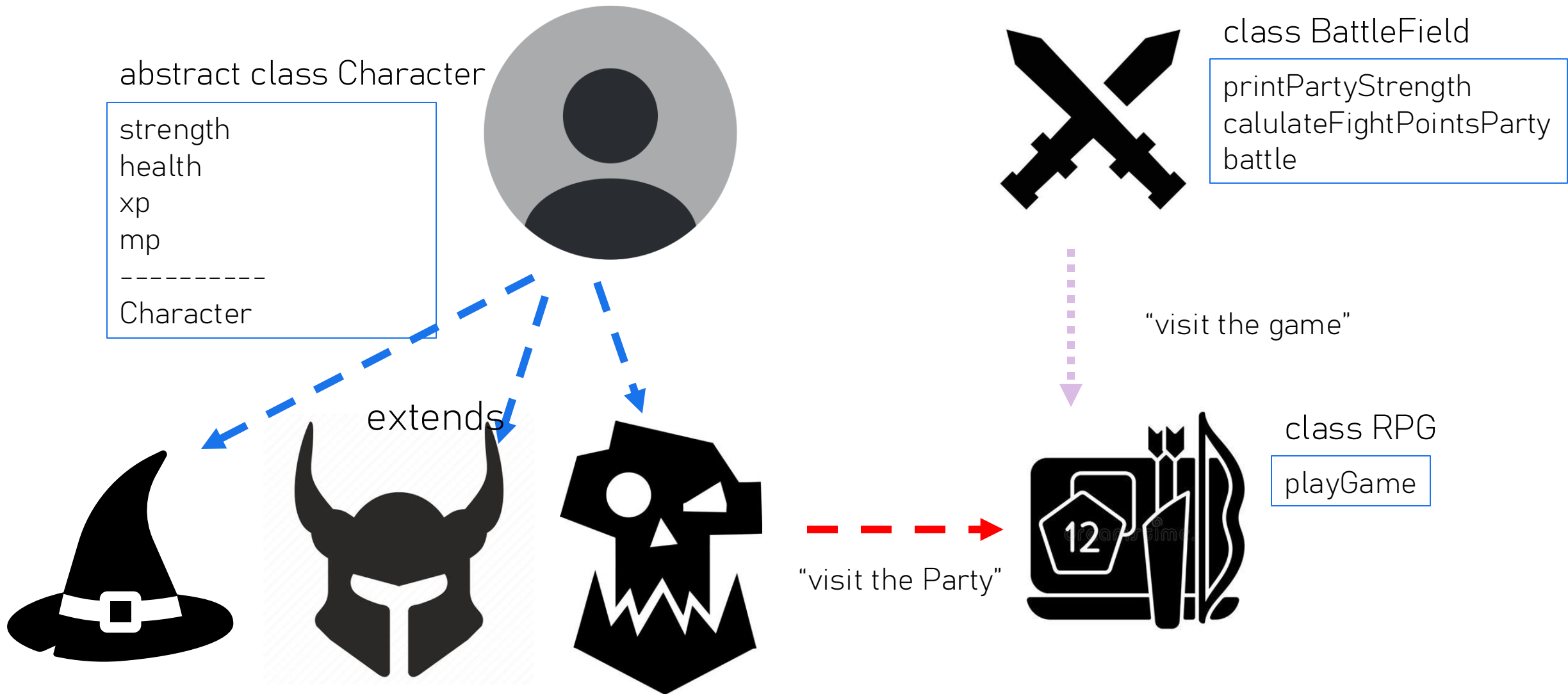
Arv vs Interface



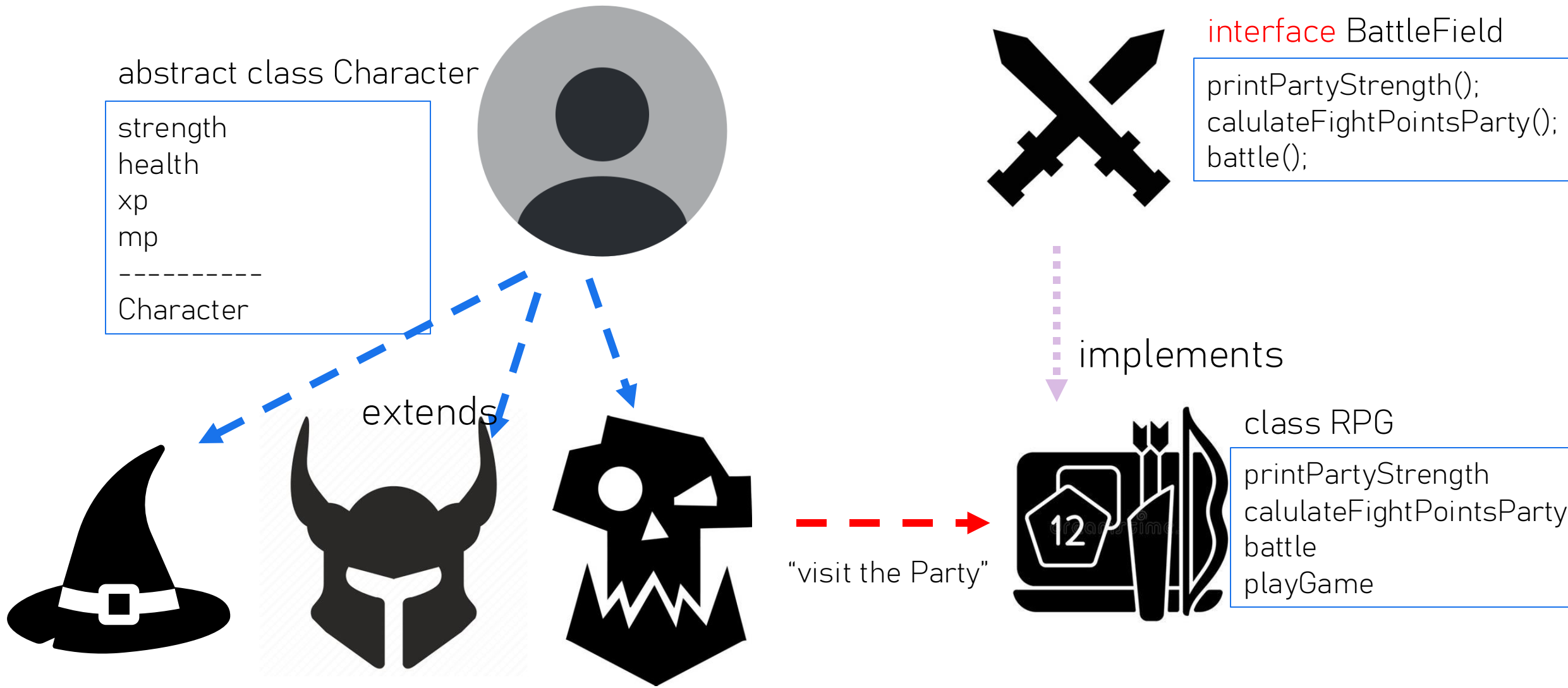
Arv vs Interface



Arv vs Interface



Arv vs Interface

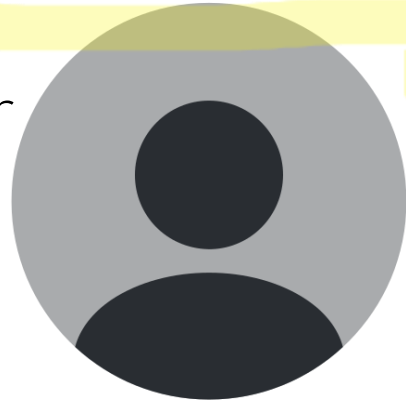


Arv vs Interface

abstract class Character

strength
health
xp
mp

Character



OBJECT
STRUCTURE

extends



interface Battlefield

printPartyStrength();
calulateFightPointsParty();
battle();



implements

ALGORITHMS
ON OBJECTS

class RPG

printPartyStrength
calulateFightPointsParty
battle
playGame



“visit the Party”

Arv vs Interface

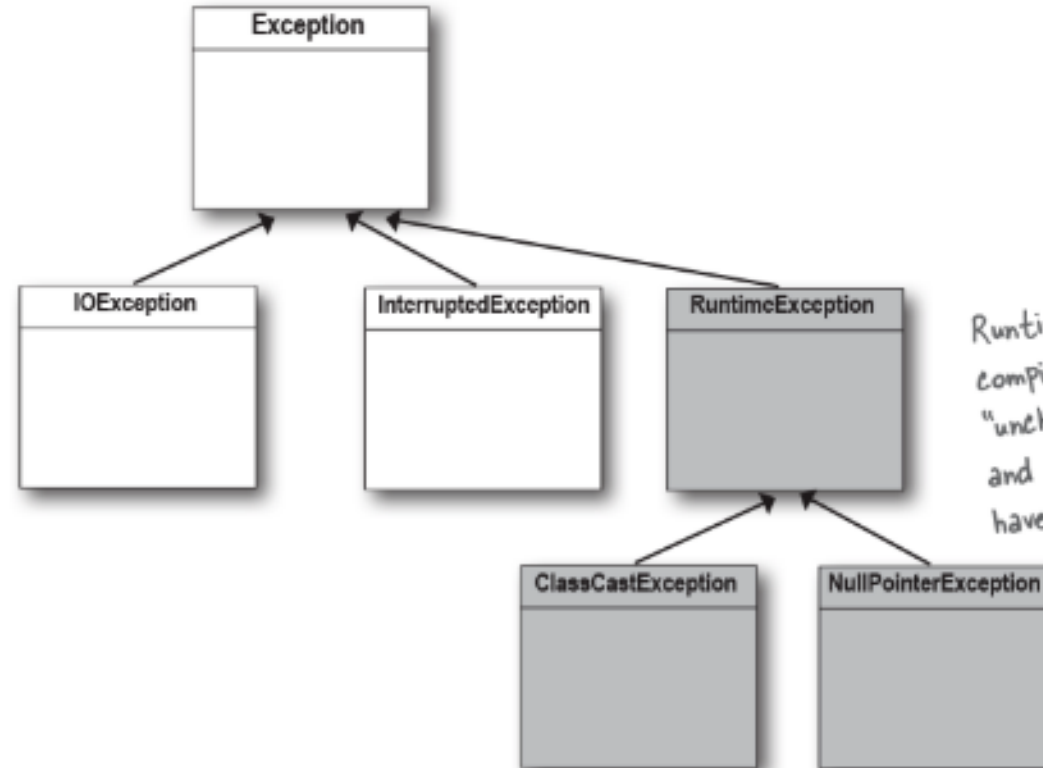
- I arv “utvides” (extends) **en** klasse til underklasser
- -> høy sammenhørighet (high cohesion)
- Interface “implementer” (implements) til andre klasser
- -> lav kopling (low coupling)

Unntakshåndtering

The compiler checks for everything except RuntimeExceptions.

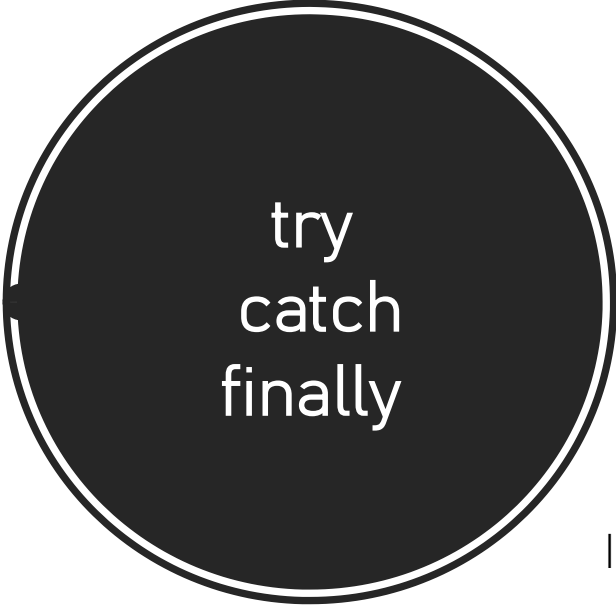
Checked vs
unchecked
exceptions

Exceptions that are *NOT* subclasses of RuntimeException are checked for by the compiler. They're called "checked exceptions."



RuntimeExceptions are *NOT* checked by the compiler. They're known as (big surprise here) "unchecked exceptions." You can throw, catch, and declare RuntimeExceptions, but you don't have to, and the compiler won't check.

Unntakshåndtering



try
catch
finally

```
try {  
    turnOvenOn();  
    x.bake();  
} catch (BakingException e) {  
    e.printStackTrace();  
} finally {  
    turnOvenOff();  
}
```

Ingen garbage-collection for ressursen:

Classes which utilize non-memory resources should provide ways to explicitly allocate/deallocate those resources. We need to explicitly call `close()` methods for deallocation of file descriptors in `finally{}`, as it will execute whether or not an exception is thrown.

Unntakshåndtering



try-
with-
resources

The `try-with-resources` statement is a `try` statement that declares one or more resources. A *resource* is an object that must be closed after the program is finished with it. The `try-with-resources` statement ensures that each resource is closed at the end of the statement. Any object that implements `java.lang.AutoCloseable`, which includes all objects which implement `java.io.Closeable`, can be used as a resource.

<https://docs.oracle.com/javase/tutorial/essential/exceptions/tryResourceClose.html>

Repetisjon : JDBC

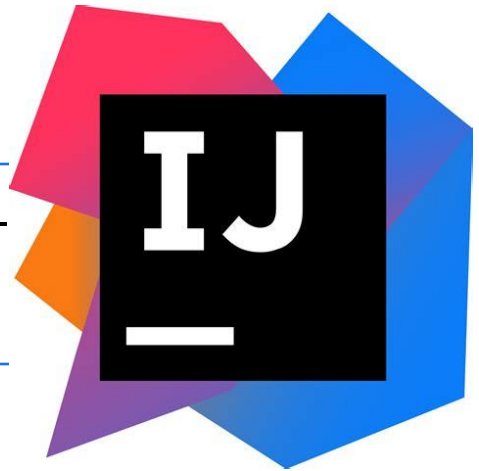
Properties: Brukes for å unngå "hard-coding" av host, db-navn, port bruker og password.

MysqlDataSource: Med denne rettes det opp forbindelse ved hjelp av properties.

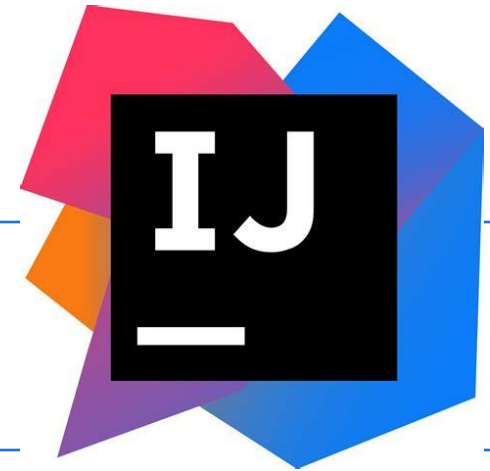
Connection: Den åpner, og lukker forbindelser til databasen. Kan brukes "under forbehold" (transaksjoner).

SQL statements: SQL som kan formateres med variabler.

Prepared statements: Bruk denne for å unngå SQL-injections.



JDBC-cont



1) autoincrement

2) Transaksjoner :

commit

rollback



»coding-eksempel» mini-RPG



Eksamen

Eksamen 2024

Du vil se at eksamensoppgaven 2024 var ganske annerledes enn i 2023, blant annet:

- Det kommuniseres med kun én database (i motsetning til to i 2023).
- Studenten fikk utlevert et databasescript for å lage databasen.
- Studenten fikk utlevert en fil med data som måtte parses.

For studentene i 2025 vil eksamen likne mer på oppgaven gitt i 2024 enn i 2023.

Fra Canvas



Eksamen



Tips:

- Viktig: ikke nok å lagre subklasser for å vise arv. Du må også lagre og bruke objekter for å subklassene (!). Derfor er det bra å bruke "abstrakt" superklasse.
- Jobb selv med filhåndtering og JDBC. Det er mye gjenbrukbart kode du kan generere her (slik som drive manager, MySQLDataSource, properties, fillesing osv). Men du må sette opp selv på din egen system.
- Anbefales å jobbe ilag med eksamen 2024 denne uken. Dette er en veldig god forberedelse til eksamen !!

Eksamen

Noen råd

- Les hele oppgaven godt før du begynner.
- Ha frokosten ferdigspist før kl 10.
- Lag en overordnet plan om hvordan du vil angripe oppgaven
 - Hvilke klasser trenger du?
 - Hvilken rekkefølge vil du lage dem?
 - Hvilke ressurser har du tilgjengelig (tidligere oppgaver du har løst etc)
- Test hyppig. Det er vanskeligere å finne feil hvis du tester sjeldent.
- Har du noen gode råd til dine medstudenter?



Eksamen

Plan for 2024-eksamen:

FileHandler: Lese fra funn.txt.

Klasse som kommuniserer med DB. Oppdatere DB med data fra filereader. Hente ut data.

Personer, museer, funn (foreldre) smykke, våpen, mynt, propertiesprovider,

Program: en klasse som tar deg gjennom noen menyvalg du kan velge mellom

Main

